

8 - WEEK REPORT

Weather Station Project

AIoT Multi-Modal Sensing Overview



Summer Research Fellowship Programme (SRFP)
Progress Report – July 2026

Submitted by:
SHAIK HAIDAR
Application No: ENGS5618
Electronics and Communication
Department
UCEK-JNTUK (Autonomous),
Kakinada

Under the Supervision of:
Dr. T V PRABHAKAR
I ISc-DESE,
Bangalore

Contents

1 Introduction	4
2 Summary of First 4 Weeks of Internship	5
3 About the Weather Station	5
3. Project Overview..... Objectives.....	6
1 Implemented System Architecture	6
3. 3.3.1 Sensors used..... 3.3.2.....	6
2 Raspberry Pi 4 (Data Acquisition and Local Processing) 3.3.3.....	7
3. MQTT Communication Layer 3.3.4 Mosquitto MQTT.....	8
3 Broker 3.3.5 CSV Data Logging..... 3.3.6.....	8
Dashboard Visualization	8
3.3.7 End Users..... Overall Data Flow.....	9
.....	9
.....	9
3.4	9
4 Work Done	10
4.1 Analysis of Weather Station State Architecture	10
4.2 Literature Review of Optimization Techniques	11
4.3 Development of the Weather GPT Framework	12
4.4 Integration of Large Language Models	13
4.5 Development of User Interfaces and Application Framework	14
4.6 Web-Based Dashboard.....	14
4.7 Command-Line Interface	14
4.8 Model Context Protocol (MCP) Integration	15
4.9 Cloud Deployment Architecture	15
4.10 System Integration.....	15
4.11 Training of the Chatbot for Weather Prediction	16
4.11.1 Model's Algorithm used	16
4.11.2 Data Preparation.....	16
4.11.3 Loss Function.....	16
4.11.4 Training Process for prediction	17
4.12 Features of the LSTM Trained Bot	18
4.12.1 Historical Weather Information Retrieval	18
4.12.2 Statistical Weather Analysis	18
4.12.3 Multi-Date Weather Comparison	20
4.12.4 LSTM-Based Weather Forecasting	20
4.12.5 Natural Language Conversational Interface	21
4.12.6 Interactive Web-Based Dashboard	21
4.12.7 Offline Operation.....	22
4.12.8 Automatic Fallback Response Generation	22
4.12.9 Modular and Scalable Architecture	22
4.13 evaluation metrics of the bot	22
4.14 Design of Bot using New Algorithm	23

5	Implemented WeatherBOT Enhancements	24
5.1	SystemArchitectureandGNNNodeForecasting..	24
5.2	KeyFeatures..... 5.3 Bot Performance and..	24
	Inference Metrics 5.4..	25
	GUIWalkthroughandVerification 5.5..	25
	evaluationmetricsoftheGNNbasedbot.....	29
		30
6	Conclusion	31
7	References	

Abstract

This work presents WeatherGPT, an AI-assisted weather monitoring and forecasting framework that integrates Artificial Intelligence (AI), Internet of Things (IoT), edge computing, and conversational intelligence into a unified environmental monitoring platform. The system employs a Raspberry Pi 4 integrated with multiple environmental sensors to continuously monitor temperature, humidity, atmospheric pressure, particulate matter, rainfall, wind speed, and wind direction. Sensor observations are processed at the edge, transmitted using the MQTT communication protocol, archived for historical analysis, and visualized through an interactive dashboard. A custom Natural Language Processing (NLP) pipeline enables users to retrieve historical weather information, perform statistical analysis, compare weather conditions across multiple dates, and interact with the system using natural language.

Short-term weather forecasting is performed using a stacked Long Short-Term Memory (LSTM) neural network implemented in PyTorch and trained on historical weather observations to generate multivariate forecasts for up to 30 days. The framework further integrates locally hosted Large Language Models (LLMs), REST APIs, command-line and web-based interfaces, and Model Context Protocol (MCP) support to provide reliable offline operation and seamless interoperability with external AI applications. Building upon this foundation, the research is being extended towards the development of an advanced WeatherBot based on Graph Neural Networks (GNNs) to model spatial and temporal relationships among multiple sensing nodes, enabling scalable multi-node weather prediction and intelligent environmental analysis for next-generation AIoT weather monitoring systems.

Keywords: WeatherGPT, Artificial Intelligence (AI), Natural Language Processing (NLP), Conversational AI, Weather Forecasting, Historical Weather Data Retrieval, Time Series Forecasting, Long Short-Term Memory (LSTM), PyTorch, Ollama, Decision Engine, Offline AI Assistant, Information Retrieval, SQLite Database, REST API, Model Context Protocol (MCP), Weather Analytics, Bangalore Weather Dataset.

1 Introduction

Environmental monitoring has become increasingly important for understanding atmospheric behavior, supporting climate studies, improving agricultural practices, enabling disaster preparedness, and facilitating the development of smart cities. Continuous observation of environmental parameters such as temperature, humidity, atmospheric pressure, particulate matter concentration, rainfall, wind speed, and wind direction provides valuable insights into changing weather conditions and environmental quality. Conventional weather monitoring systems primarily focus on data collection and visualization; however, the growing availability of Artificial Intelligence (AI), Internet of Things (IoT), and edge computing technologies has created opportunities to develop more intelligent systems capable of not only acquiring environmental data but also interpreting, analyzing, and predicting weather conditions in real time.

This project presents an (Artificial Intelligence of Things)AIoT-based Multi-Modal Weather Monitoring and Intelligent Weather Assistant System that combines an embedded weather station with a conversational AI chatbot. The weather monitoring subsystem is built around a Raspberry Pi 4 and integrates multiple environmental sensors to continuously measure key atmospheric parameters, including temperature, humidity, pressure, particulate matter, rainfall, wind speed, and wind direction. Sensor observations are processed locally at the edge and transmitted using the lightweight MQTT communication protocol through a Mosquitto Broker. The collected data are stored for long-term analysis and visualized through an interactive dashboard, providing users with real-time access to environmental information while ensuring reliable and efficient data management.

To enhance user interaction and extend the capabilities of the weather station, the project incorporates WeatherGPT (WGPT), an AI-powered weather intelligence assistant that enables users to access weather information through natural language conversations. The chatbot combines Natural Language Processing (NLP), machine learning, and Large Language Models (LLMs) to interpret user queries, retrieve historical weather observations, perform statistical analysis, and generate short-term weather forecasts. Historical weather data are obtained from locally stored datasets, while future weather conditions are predicted using a Long Short-Term Memory (LSTM) neural network trained on historical weather records. Retrieved data and prediction results are converted into conversational responses using a locally hosted Ollama-based LLM, with a rule-based response mechanism serving as a fallback to ensure uninterrupted offline operation.

The overall system follows a modular architecture that separates sensing, data acquisition, communication, storage, artificial intelligence, and user interaction into independent functional layers. This design improves scalability, simplifies maintenance, and allows additional sensors, forecasting models, and AI services to be incorporated with minimal modifications. Furthermore, the system supports multiple user interfaces, including a command-line chatbot, a web-based dashboard with REST APIs, and a Model Context Protocol (MCP) server, enabling seamless integration with external AI applications and intelligent assistants.

By integrating edge computing, IoT communication, machine learning, and conversational AI into a unified platform, the proposed system demonstrates an efficient and cost-effective approach for intelligent environmental monitoring. Beyond conventional weather stations, it provides users with an interactive environment capable of real-time monitoring, historical data exploration, and predictive weather analysis, making it well suited for research, educational, and smart environmental monitoring applications.

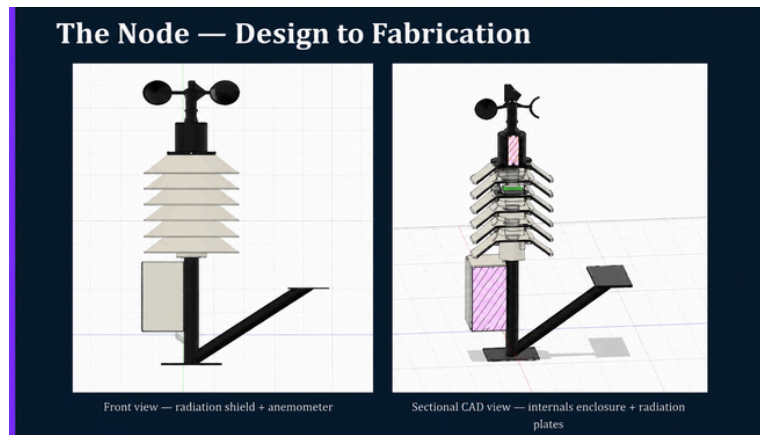


Figure 1: Model of the Weather Station node

The remainder of this report describes the system architecture, hardware and software implementation, sensor integration, communication framework, chatbot design, forecasting methodology, and experimental evaluation of the proposed AIoT-based intelligent weather monitoring system.

2 Summary of First 4 Weeks of Internship

During the first four weeks of the Summer Research Fellowship, the primary focus was on understanding the architecture and operation of an intelligent weather monitoring system. This involved studying the Raspberry Pi 4B platform, environmental sensors, communication protocols, network time synchronization, and data quality metrics such as AoI and VAOI. A comprehensive review of relevant literature was also carried out to understand the system architecture, data flow, and edge-based processing, providing a strong foundation for the project.

Based on this understanding, the scope of the project was extended by exploring potential improvements to the existing weather station. This included studying additional sensing and synchronization techniques, evaluating power-monitoring components, and investigating energy optimization approaches such as Dynamic Voltage and Frequency Scaling (DVFS) for Raspberry Pi systems. These studies established the technical groundwork for the implementation and evaluation phases planned for the remainder of the internship.

3 AbouttheWeatherStation

The proposed AIoT Multi-Modal Weather Station is designed as a comprehensive environmental monitoring platform capable of continuously sensing, processing, storing, and visualizing multiple atmospheric parameters in real time. Unlike conventional weather stations that primarily focus on a limited set of meteorological variables, this system integrates multiple sensing modalities to provide a holistic understanding of environmental conditions. The architecture combines heterogeneous sensors, edge computing, lightweight communication protocols, local storage, and visualization techniques to create a reliable and scalable monitoring solution.

3.1 Project Overview

Environmental monitoring has become increasingly important due to rapid climate change, urbanization, and growing concerns regarding air quality and weather-related hazards. Accurate measurement of atmospheric parameters enables researchers and engineers to analyze weather patterns, monitor pollution levels, and develop intelligent environmental monitoring systems.

The proposed AIoT Multi-Modal Weather Station addresses these requirements by integrating multiple environmental sensors with a Raspberry Pi 4-based processing unit. The collected sensor readings are processed locally and transmitted using the MQTT communication protocol before being archived in CSV format for visualization and future analysis. The system is capable of continuously monitoring multiple weather parameters while maintaining a lightweight and energy-efficient architecture suitable for research and field deployments.

The modular implementation ensures flexibility in hardware expansion and software enhancements while providing a robust platform for continuous environmental monitoring.

3.2 Objectives

The primary objectives of the proposed weather monitoring station are summarized below:

- To continuously monitor multiple environmental parameters in real time.
- To integrate heterogeneous weather sensors into a unified monitoring platform.
- To perform reliable local acquisition and preprocessing of sensor observations.
- To establish lightweight communication using the MQTT publish-subscribe protocol.
- To maintain local archival of weather observations using CSV-based storage.
- To provide intuitive visualization of environmental data through a dashboard interface.
- To develop a modular and scalable AIoT architecture suitable for future research applications.

3.3 Implemented System Architecture

The implemented weather station follows a layered architecture designed to simplify data acquisition, communication, storage, and visualization. Each subsystem performs a dedicated function while collectively contributing to reliable environmental monitoring.

Table: Integrated Sensors and Their Specifications

s.no	Sensor Name	Measured Parameter(s)	Communication Interface	Purpose
1	BME280	Temperature, Relative Humidity, Atmospheric Pressure	I ² C	Environmental monitoring and weather analysis
2	SPS30	PM1.0, PM2.5, PM4.0, PM10	UART	Air quality and particulate matter monitoring
3	YL-83 Rain Sensor	Rain Presence Detection	GPIO	Detection of rainfall events
4	Tipping Bucket Rain Gauge	Rainfall Amount	I2C	Quantitative rainfall measurement
5	Anemometer	Wind Speed	GPIO	Wind speed monitoring
6	Wind Vane	Wind Direction	I2C	Wind direction estimation

Figure 2: Implemented System Architecture Diagram

The major components of the implemented architecture include:

- Sensor Layer
- Raspberry Pi 4 (Data Acquisition and Local Processing)
- MQTT Communication Layer
- Mosquitto MQTT Broker
- CSV Data Logging
- Dashboard Visualization
- End Users

The interaction among these components enables continuous monitoring of atmospheric conditions while maintaining efficient data handling and minimal communication overhead.

3.3.1 Sensors used

The Sensors forms the foundation of the AIoT weather station by continuously acquiring environmental observations from the surrounding atmosphere. Multiple sensors are integrated to capture different meteorological and air-quality parameters, thereby providing a comprehensive representation of weather conditions.

The implemented sensing devices include:

- BME280 for measuring temperature, relative humidity, and atmospheric pressure.
- SPS30 for measuring particulate matter concentrations including PM1.0, PM2.5, PM4.0, and PM10.

- YL-83 for detecting the presence of rainfall.
- Tipping Bucket Rain Gauge for measuring accumulated rainfall.
- Anemometer for monitoring wind speed.
- Wind Vane for measuring wind direction.

Each sensor communicates with the Raspberry Pi through suitable interfaces such as I²C, UART, or GPIO depending on its communication protocol. The combination of multiple sensing modalities enables comprehensive environmental monitoring under varying atmospheric conditions.

3.3.2 Raspberry Pi 4 (Data Acquisition and Local Processing)

The Raspberry Pi 4 serves as the central processing unit of the proposed weather station. It acts as the interface between the sensing layer and the communication infrastructure by periodically acquiring measurements from all connected sensors.

Upon receiving sensor observations, the Raspberry Pi performs several preprocessing operations, including sensor polling, data formatting, timestamp generation, and validation of acquired measurements. These preprocessing steps ensure that only structured and meaningful environmental information is forwarded to subsequent stages of the system.

Local processing minimizes computational overhead on downstream applications while improving the overall reliability and efficiency of the weather monitoring platform.

3.3.3 MQTTCommunicationLayer

Following local processing, the acquired environmental observations are transmitted using the Message Queuing Telemetry Transport (MQTT) protocol. MQTT is a lightweight publish-subscribe communication protocol specifically designed for Internet of Things applications requiring minimal bandwidth consumption and reliable communication.

Within the implemented architecture, the Raspberry Pi functions as the MQTT Publisher by periodically publishing processed weather observations to predefined communication topics. The publish-subscribe model eliminates direct coupling between data producers and consumers, thereby improving scalability and communication efficiency.

3.3.4 MosquittoMQTTBroker

The Mosquitto Broker functions as the communication hub of the proposed system. It receives MQTT messages published by the Raspberry Pi and manages their routing toward the storage subsystem.

The broker is responsible for:

- Receiving published environmental observations.
- Managing MQTT communication topics.
- Forwarding weather data to subscribed applications.
- Ensuring reliable message delivery throughout the communication process.

The use of Mosquitto simplifies communication management while enabling efficient data exchange between hardware and software components.

3.3.5 CSV Data Logging

To preserve environmental observations for future reference, all incoming weather data are archived locally as CSV files.

CSV-based storage provides a lightweight and platform-independent mechanism for maintaining historical environmental records. This approach enables efficient retrieval of previously collected observations while simplifying data analysis and visualization.

The local storage mechanism also serves as a backup repository, ensuring that valuable sensor data remain available even during temporary communication interruptions.

3.3.6 Dashboard Visualization

The visualization dashboard provides a user-friendly graphical interface for observing environmental conditions in real time.

The dashboard continuously retrieves stored weather observations from the CSV files and presents them through graphical charts and numerical indicators. The visualization interface enables users to monitor current weather conditions while simultaneously analyzing historical environmental trends.

The dashboard visualizes the following parameters:

- Temperature, Relative Humidity, Atmospheric Pressure
- PM1.0, PM2.5, PM4.0, PM10
- Rain Detection, Rainfall Amount
- Wind Speed, Wind Direction

The visualization platform significantly improves the interpretability of collected weather information and supports efficient environmental monitoring.

3.3.7 End Users

The final component of the implemented architecture consists of the end users who interact with the weather monitoring station through the visualization dashboard. The dashboard enables these users to continuously monitor environmental conditions, analyze historical observations, and evaluate overall system performance through an intuitive graphical interface.

3.4 Overall Data Flow

The operational workflow of the implemented AIoT Multi-Modal Weather Station is illustrated in Figure 3. The data flow begins with the environmental sensors, which continuously acquire atmospheric observations. These measurements are transferred to the Raspberry Pi 4, where local data acquisition and preprocessing are performed. The processed sensor readings are then published using the MQTT protocol and forwarded to the Mosquitto MQTT Broker. Rather than storing the observations in a remote database, the proposed implementation maintains local CSV-based storage, where all environmental measurements are archived for future retrieval. The visualization dashboard periodically reads the stored CSV files and presents the information through interactive graphical

interfaces. Finally, researchers, developers, and system operators access the dashboard to observe both real-time and historical environmental conditions.

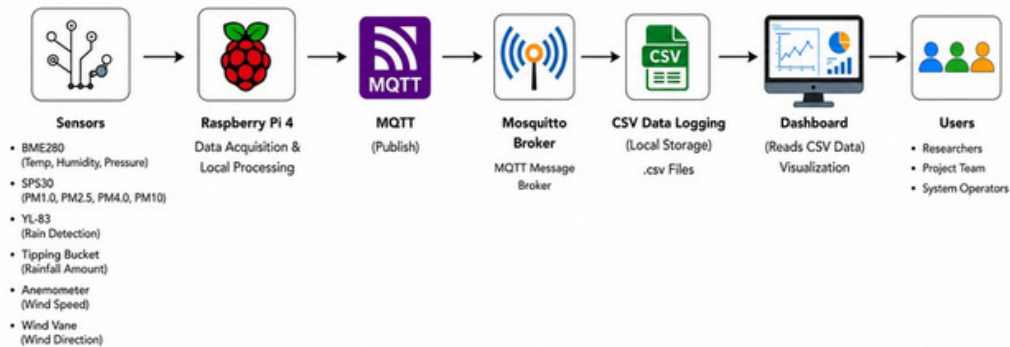


Figure 3: Data Flow of the Weather Station

4 WorkDone

4.1 AnalysisofWeatherStationStateArchitecture

The initial phase of the second half of the internship focused on understanding and evaluating the operational state architecture of the weather station. Before introducing artificial intelligence and predictive capabilities into the system, it was necessary to understand how the existing weather station monitored environmental conditions and generated alerts based on sensor observations.

The functionality of each operational state was examined by studying the corresponding transition conditions, threshold values, and alert generation logic. Particular attention was given to understanding how changes in meteorological parameters triggered transitions between different operational states and how these transitions influenced the decision-making process of the monitoring system.

Different state configurations were evaluated conceptually by considering possible scenarios such as gradual environmental changes, sudden fluctuations in weather conditions, and sensor noise. The analysis also included studying the impact of introducing additional alerting states on system complexity, computational overhead, maintainability, and interpretability.

The complete workflow of the weather monitoring station can be summarized as follows:

- Environmental sensors continuously monitor atmospheric conditions basing on their sampling rates.
- The Raspberry Pi 4 acquires and locally processes the sensor observations.
- Processed weather data are published using the MQTT protocol.
- The Mosquitto MQTT Broker receives and manages the published messages.

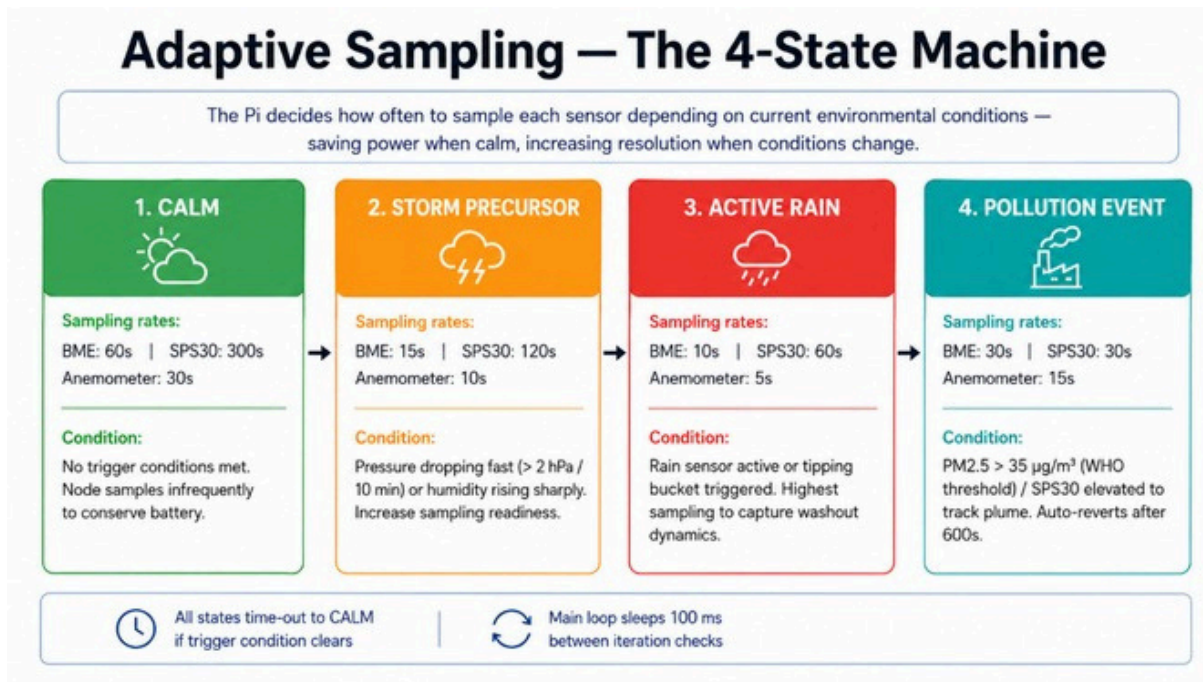


Figure 4: overview of Machine Learning Model Alternatives

- Sensor observations are archived locally in CSV files.
- The dashboard retrieves weather information from the CSV files.
- End users visualize and analyze the environmental observations through the dashboard.

. Therefore, the existing state model in figure 4 was retained as the foundation for the subsequent development of the AI-assisted weather prediction framework. The knowledge acquired during this phase provided a comprehensive understanding of the weather station's operational workflow and served as a prerequisite for integrating machine learning and conversational intelligence into the existing architecture.

4.2 Literature Review of Optimization Techniques

Following the analysis of the weather station architecture, an extensive literature review was conducted to investigate recent advancements in weather prediction systems, optimization methodologies, and artificial intelligence techniques applicable to meteorological forecasting.

Research papers from journals, conference proceedings, and technical publications were reviewed to understand the evolution of weather forecasting methods, ranging from conventional statistical approaches to modern machine learning and deep learning techniques. The review primarily focused on methodologies used for time-series prediction, sequence modelling, feature engineering, optimization strategies, and intelligent decision- support systems.

Particular emphasis was placed on identifying optimization techniques capable of improving prediction accuracy while maintaining computational efficiency. Various studies

were examined to understand the influence of data preprocessing, feature normalization, hyperparameter tuning, learning rate optimization, loss function selection, and model regularization on forecasting performance. Different optimization algorithms and training strategies reported in the literature were also compared to understand their advantages and limitations in weather prediction applications.

In addition to optimization methods, the literature survey included a comparative study of several predictive algorithms commonly used in meteorological forecasting, including Linear Regression, Decision Trees, Random Forests, Support Vector Machines (SVM), Artificial Neural Networks (ANN), Recurrent Neural Networks (RNN), Gated Recurrent Units (GRU), and Long Short-Term Memory (LSTM) networks. Each algorithm was evaluated based on its ability to model nonlinear relationships, capture temporal dependencies, process sequential weather observations, and generate accurate long-term forecasts.

The review indicated that recurrent deep learning architectures consistently outperformed conventional machine learning models when applied to multivariate weather time-series datasets. Among these, the Long Short-Term Memory (LSTM) network demonstrated superior capability in learning long-range temporal dependencies while effectively addressing the vanishing gradient problem encountered in conventional recurrent neural networks. Later, due to its simplicity in designing, it was decided to choose the LSTM algorithm to develop a weather predicting chatbot.

Overall, the literature review established the theoretical foundation for the project by identifying state-of-the-art forecasting methodologies, evaluating existing optimization techniques, and providing a systematic basis for the architectural and algorithmic decisions made during the development of the intelligent weather prediction system.

4.3 Development of the WeatherGPT Framework

Following the selection of the LSTM model and the overall system architecture, the implementation focused on developing a complete AI-assisted weather prediction framework, referred to as WeatherGPT. The objective was to integrate natural language understanding, machine learning-based weather forecasting, and conversational artificial intelligence into a unified system capable of providing accurate and user-friendly weather information.

The first stage of development concentrated on designing the intelligence layer of the WeatherGPT framework. This layer was responsible for understanding user queries, retrieving relevant weather information, generating forecasts, and producing natural language responses.

A custom Natural Language Processing (NLP) pipeline was developed to process user queries efficiently. The pipeline consisted of a tokenizer, an intent classification module, and an entity extraction mechanism. The tokenizer segmented user input into meaningful tokens, while the intent classifier identified the purpose of the query, such as requesting weather information for a specific date, obtaining statistical summaries, or generating future weather forecasts. The entity extraction module identified temporal information, including dates and time periods, enabling the system to retrieve relevant weather records accurately.

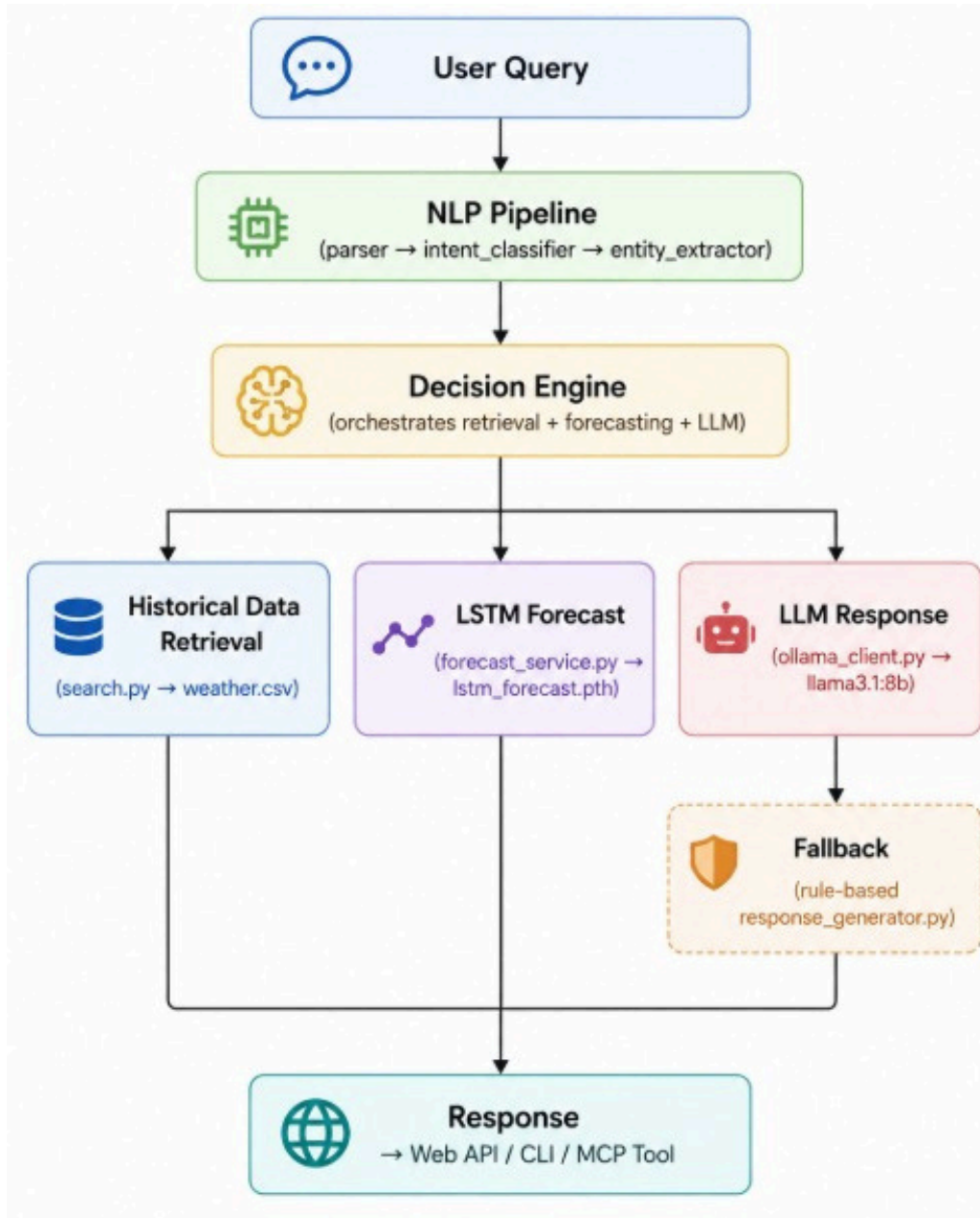


Figure 5: Architecture of the LSTM Weather Chatbot

In situations where neither language model was accessible, a rule-based response generation module ensured uninterrupted system operation. This fallback architecture significantly improved system reliability and fault tolerance.

4.4 Integration of Large Language Models

To provide conversational responses, the prediction engine was integrated with a Large Language Model (LLM). Initially, Ollama 3.1 (8B) was employed for local inference during development. Subsequently, Groq-based inference was incorporated to enable faster response generation and improved conversational performance.

To enhance the robustness of the application, a multi-level response generation mechanism was implemented. If the primary language model was unavailable, the system

automatically switched to an Ollama-based inference engine.

4.5 Development of User Interfaces and Application Framework

After validating the functionality of the core processing engine, the next phase involved transforming the prediction framework into a complete software application that could be accessed through multiple interfaces.

4.6 Web-Based Dashboard

A web-based dashboard was developed to provide an intuitive interface for interacting with the WeatherGPT system. A lightweight RESTful API was implemented using Python-based web technologies to expose endpoints responsible for chatbot interaction and weather prediction services.

The frontend was developed as a Single-Page Application (SPA) using HTML, CSS, and JavaScript. The dashboard enabled users to interact with the chatbot through a conversational interface while simultaneously displaying system information and weather prediction results. Interactive visualizations were incorporated to present forecasted weather trends graphically, thereby improving the interpretability of prediction outputs.

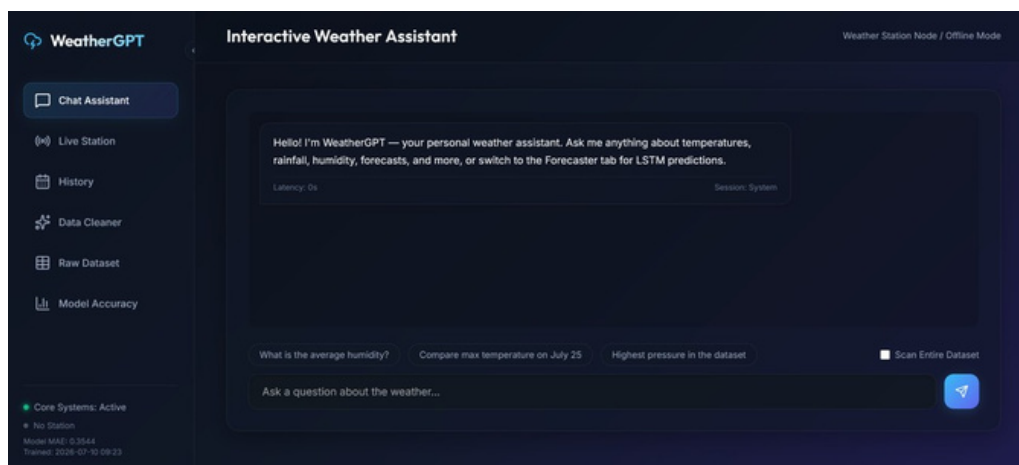


Figure 6: Vite + React Dashboard Frontend Screenshot

4.7 Command-Line Interface

In addition to the graphical web interface, a command-line application was developed to facilitate direct interaction with the WeatherGPT framework from the terminal. This interface proved particularly useful during system testing, debugging, and rapid experimentation.

To simplify execution across different development environments, an automated environment detection mechanism was implemented. The application verified whether the correct Python virtual environment was active and, if necessary, automatically relaunched itself within the configured virtual environment. This approach minimized dependency-related execution errors and improved the portability of the application.

4.8 ModelContextProtocol(MCP)Integration

To facilitate seamless integration between the AI-powered chatbot and the AIoT-based weather monitoring station, the Model Context Protocol (MCP) was adopted as the communication framework.

MCP provides a standardized protocol that enables AI applications to securely interact with external tools, services, and data sources through a unified interface. By eliminating the need for custom integration logic, MCP simplifies communication between the chatbot and the weather station while improving interoperability and maintainability.

In the proposed system, an MCP-compatible server was developed using the FastMCP framework to expose the weather station's functionalities as callable tools. Through this server, the chatbot can communicate directly with the weather station to access real-time sensor observations, retrieve historical weather records, and invoke weather-related services using standardized MCP requests. The MCP server acts as an intermediary between the chatbot and the weather station, managing request routing, tool execution, and response delivery in a structured and consistent manner.

The adoption of MCP enhances the modularity, scalability, and extensibility of the overall architecture by providing a common communication layer that decouples the chatbot from the underlying weather station implementation. This approach allows additional AI applications, intelligent agents, or external development environments supporting MCP to integrate with the weather station without requiring modifications to the existing system. Consequently, the weather station can be utilized as a reusable AI service, enabling efficient access to environmental data and weather-related functionalities through a standardized protocol while supporting future expansion of the system.

4.9 Cloud Deployment Architecture

The final stage of development focused on deployment and accessibility. The complete application, including the backend server, REST API, prediction engine, chatbot, and frontend dashboard, was configured for cloud deployment.

Deployment configuration files were created to enable seamless hosting on cloud platforms such as Render. This deployment architecture ensured that the WeatherGPT framework could be executed as an online application without requiring extensive manual configuration. The modular deployment strategy also facilitates future scalability, maintenance, and integration with real-time weather data sources.

4.10 System Integration

Upon completion of all development phases, the individual components—including the NLP pipeline, historical weather database, LSTM prediction model, conversational language model, retrieval engine, web dashboard, command-line interface, and MCP server—were integrated into a unified WeatherGPT framework.

The resulting system provides end-to-end functionality by accepting natural language weather queries, retrieving relevant historical information, generating AI-based weather forecasts, and presenting results through multiple user interfaces. This integrated architecture demonstrates the practical application of deep learning, natural language processing, and modern software engineering principles in the development of an intelligent weather forecasting assistant.

4.11 Training of the Chatbot for Weather Prediction

The weather prediction component of WeatherGPT was implemented using a Stacked Long Short-Term Memory (LSTM) neural network developed in the PyTorch deep learning framework. The model was designed to capture temporal dependencies present in sequential weather observations and generate accurate multivariate weather forecasts based on historical weather data.

During the development and validation of the chatbot, mock weather readings were used to train, validate, and verify the prediction pipeline. These simulated datasets enabled testing of the chatbot's forecasting functionality and ensured that it could correctly process user queries and generate weather predictions before deployment with live weather station data. By learning patterns from historical weather observations, the model is capable of predicting multiple meteorological parameters for a forecasting horizon of up to 30 days. Multiple training iterations were performed to optimize the network parameters and improve the overall predictive performance. Upon completion of training, the optimized model was integrated into the WeatherGPT framework, enabling real-time weather forecasting during chatbot interactions.

4.11.1 Model's Algorithm used

The forecasting model is based on a Stacked LSTM architecture consisting of two LSTM layers, each with 64 hidden units. The stacked configuration enables the network to learn both short-term and long-term temporal relationships within historical weather sequences. A selective skip connection is incorporated to preserve the most recent values of relatively stable weather variables, thereby improving prediction stability. Furthermore, a dropout rate of 0.2 is applied between the LSTM layers to reduce overfitting and enhance the model's generalization capability.

4.11.2 Data Preparation

Prior to training, the historical weather dataset underwent preprocessing to improve the learning capability of the model. Feature engineering techniques, including cyclical encoding of time-based variables and rolling statistical features, were applied to capture seasonal patterns and recent weather trends. The processed data was then normalized using StandardScaler, with separate scalers maintained for the input features and target variables. Finally, the dataset was divided sequentially into 90% training data and 10% validation data, preserving the chronological order of observations for realistic model evaluation.

4.11.3 Loss Function

To optimize multivariate weather prediction, the model employed a custom Weighted L1 Loss (Weighted Mean Absolute Error) function. Since multiple weather variables are predicted simultaneously, different static weights were assigned to individual output parameters according to their importance. Higher weights were allocated to critical variables such as temperature and humidity, enabling the model to prioritize accurate prediction of the most significant weather metrics.

4.11.4 Training Process for prediction

The training process involved repeatedly feeding sequential weather observations into the stacked LSTM network while minimizing the weighted prediction error. During each epoch, model parameters were updated using the Adam optimization algorithm.

The model was trained using a predefined set of hyperparameters to achieve efficient convergence and reliable forecasting performance. A lag window of five days was used, allowing the network to learn from the weather observations recorded during the previous five days. The training process was carried out for a maximum of 200 epochs using a batch size of 64. Model parameters were optimized using the Adam optimizer with a learning rate of 0.003, enabling stable and efficient learning throughout the training process. To prevent overfitting, an Early Stopping mechanism with a patience of 25 epochs was incorporated. If the validation loss failed to improve for 25 consecutive epochs, the training process terminated automatically, and the model restored the weights corresponding to the best-performing validation epoch.

Table 1: MAE Readings of Mock Data Used

Weather Feature	MAE (Original Units)
temp_max	1.0404 °C
temp_min	0.4678 °C
temp	0.4534 °C
feelslike	0.5955 °C
humidity	2.5185 %
dew	0.3734 °C
precip	6.0472 mm
precip_prob	40.2123 %
windgust	12.0691 km/h
windspeed	20.1067 km/h
winddir	85.1951 degrees
pressure	0.5888 hPa
cloudcover	15.6895 %
visibility	0.9058 km
uvindex	1.9462 rating

This strategy ensured that the final model achieved optimal prediction performance without unnecessary training.

The system also supports background retraining, allowing the prediction model to be updated with newly available weather observations without interrupting chatbot operation. By default, background retraining is performed for 150 epochs, ensuring that the forecasting model remains adaptive to newly collected environmental data.

Features of the LSTM Trained Bot

4.12.1 Historical Weather Information Retrieval

WeatherGPT enables users to retrieve historical weather information using natural language queries. The chatbot accesses a locally stored weather dataset and provides observations for parameters such as temperature, humidity, atmospheric pressure, precipitation, wind speed, wind direction, cloud cover, visibility, and other environmental variables. Users can request weather information for specific dates or custom date ranges without requiring knowledge of database queries or file structures.

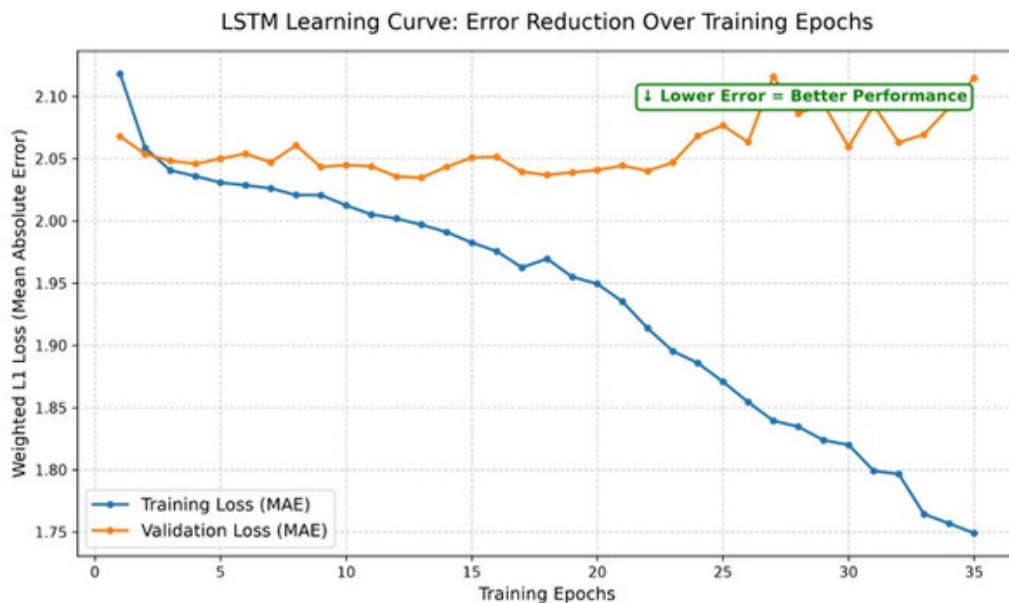


Figure 7: Historical Weather Information Retrieval Screenshot

4.12.2 Statistical Weather Analysis

The chatbot supports statistical analysis of historical weather data by computing descriptive measures, including average, minimum, and maximum values for selected weather parameters. Statistical summaries can be generated for individual days, months, or user-defined periods, enabling efficient analysis of weather trends and seasonal variations through simple conversational interactions.

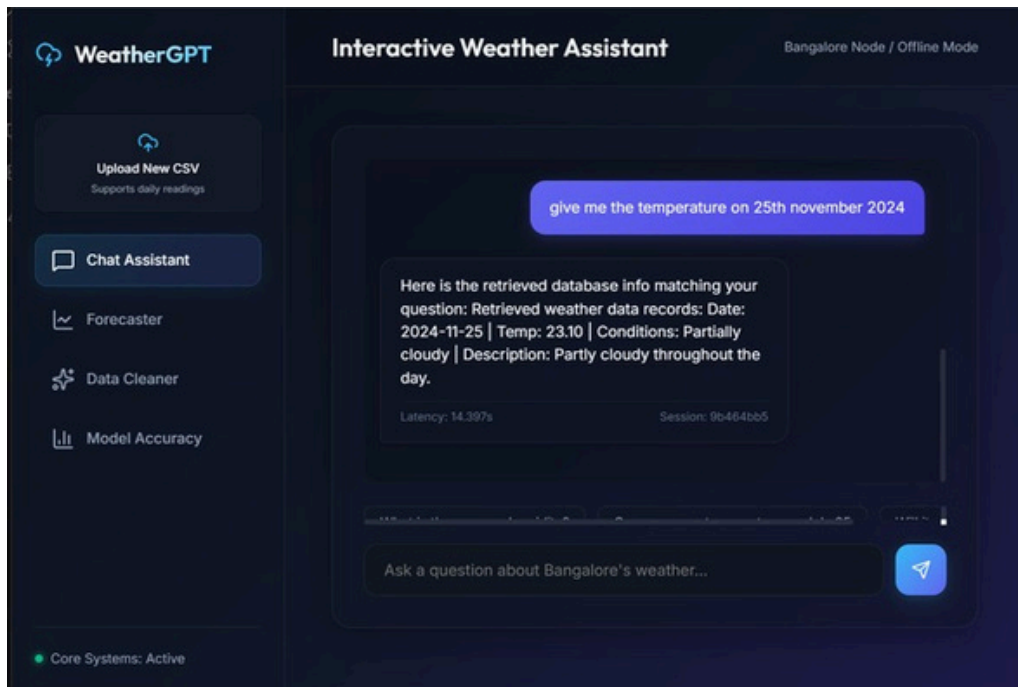


Figure 8: Statistical Analysis Output Panel

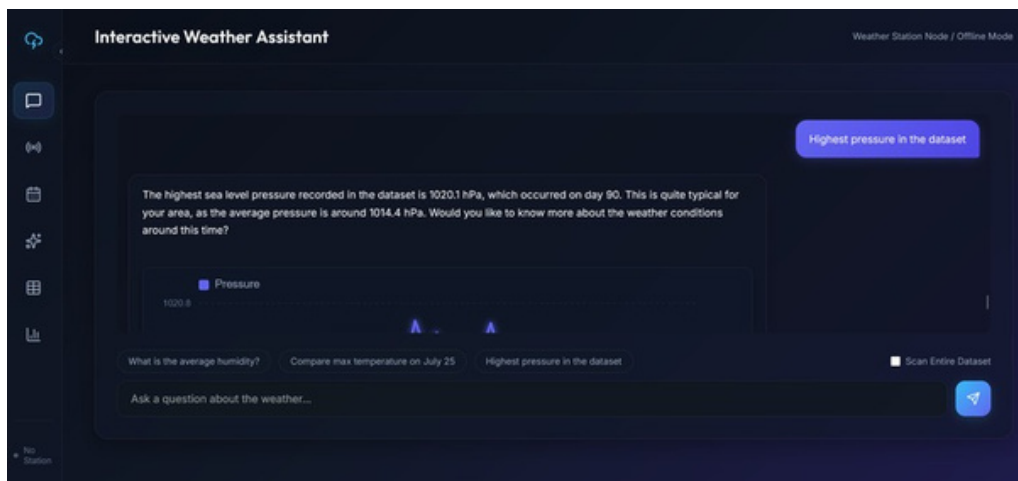


Figure 9: Statistical Analysis Secondary Chart



Figure 10: Statistical Analysis Table Details

4.12.3 Multi-DateWeatherComparison

WeatherGPT provides comparative analysis of weather conditions across multiple dates. Users can compare environmental parameters recorded on different days to identify variations in atmospheric conditions, making the system useful for weather trend analysis and environmental studies.

4.12.4 LSTM-Based Weather Forecasting

To provide predictive weather intelligence, the chatbot incorporates a stacked Long Short- Term Memory (LSTM) neural network developed using the PyTorch framework. The forecasting model utilizes historical weather observations to predict future weather conditions for up to 30 days. Forecast results are presented through natural language responses, allowing users to obtain short-term weather predictions without directly interacting with the underlying machine learning model.

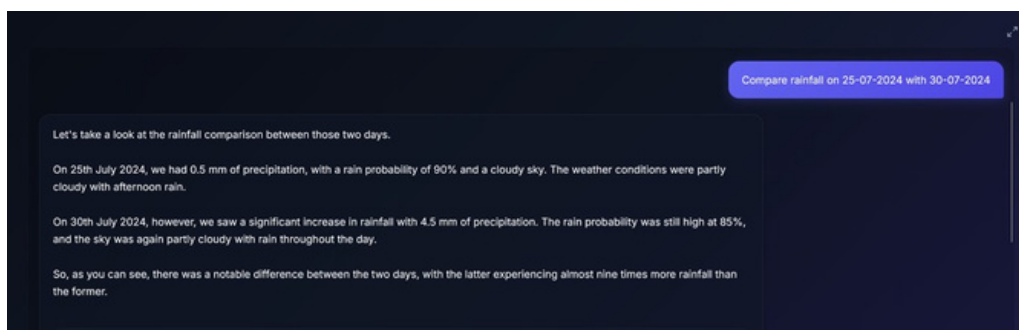


Figure 11: LSTM Forecasting Module Output



Figure 12: LSTM Forecasting Predictions Graph

4.12.5 NaturalLanguageConversationalInterface

The chatbot employs a Natural Language Processing (NLP) pipeline consisting of query parsing, intent classification, entity extraction, and decision-making modules. This enables users to communicate using everyday language rather than predefined commands, providing a more intuitive and user-friendly method for accessing weather information.

4.12.6 Interactive Web-Based Dashboard

In addition to the command-line interface, WeatherGPT provides a web-based dashboard for visualizing weather information. The dashboard presents historical observations, forecast results, interactive charts, and dataset health statistics, enabling users to explore environmental data through a graphical interface.

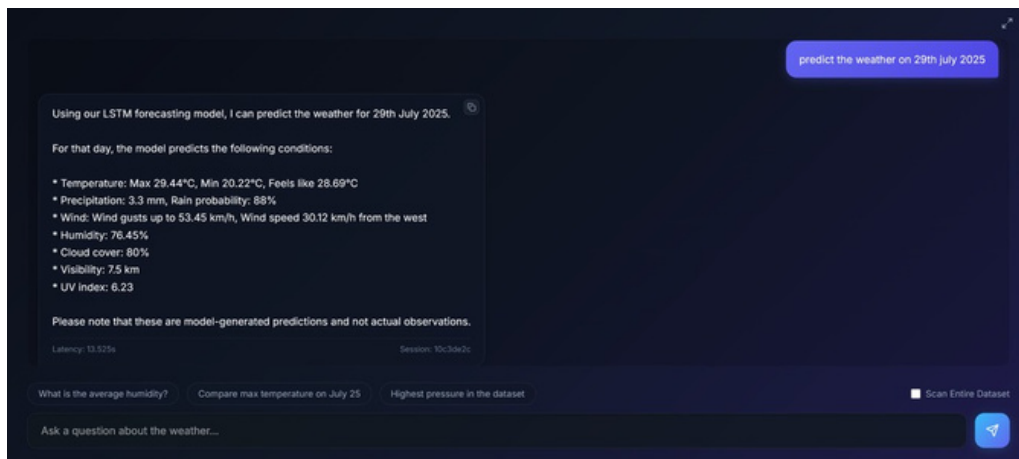


Figure 13: Interactive Charts on Web Dashboard

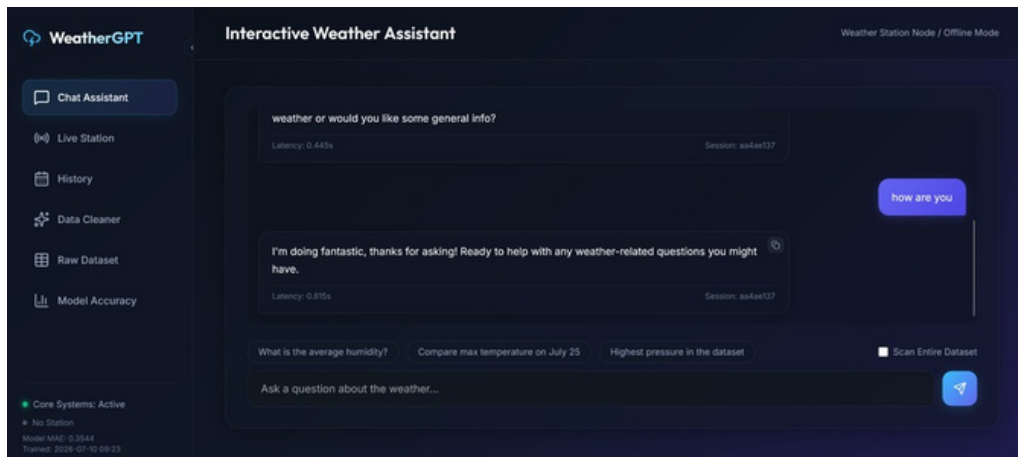


Figure 14: Dataset Health Statistics on Web Dashboard

4.12.7 OfflineOperation

The chatbot is designed to operate in offline mode as well, using locally stored datasets and locally hosted software components. This capability makes the system suitable for deployment on edge computing platforms such as the Raspberry Pi, where reliable internet connectivity may not always be available.

4.12.8 AutomaticFallbackResponseGeneration

To improve system reliability, WeatherGPT incorporates an automatic fallback mechanism that activates whenever the locally hosted Large Language Model (LLM) is unavailable. In such situations, the chatbot generates structured rule-based responses, ensuring uninterrupted operation and consistent user interaction without compromising the availability of weather services.

4.12.9 ModularandScalableArchitecture

WeatherGPT follows a modular software architecture in which the NLP pipeline, decision engine, forecasting module, response generation module, web interface, and MCP server are implemented as independent functional components. This modular design simplifies maintenance, facilitates future enhancements, and enables seamless integration of additional weather services, machine learning models, or AI applications with minimal modifications to the existing framework.

4.13 evaluation metrics of the bot

To evaluate the effectiveness of the WeatherGPT chatbot in understanding user requests, the Natural Language Processing (NLP) intent classification module was tested using a dataset of 200 representative weather-related queries. The evaluation measured the chatbot's ability to correctly identify user intents while minimizing incorrect classifications. Experimental results showed that the chatbot correctly classified 182 user queries as the intended weather-related requests (True Positives) while correctly rejecting 10 irrelevant or unsupported queries (True Negatives). Only three queries were incorrectly classified as valid intents (False Positives), and five relevant queries were not recognized

Metric	Value
Total Test Queries	200
True Positives (TP)	182
True Negatives (TN)	10
False Positives (FP)	3
False Negatives (FN)	5
Accuracy	96.00%
Precision	98.38%
Recall	97.33%
F1-Score	97.85%
Specificity	76.92%

Figure 15: Evaluation metrics of the LSTM based bot

correctly (False Negatives). These results indicate that the developed intent classification pipeline provides highly reliable performance for conversational weather interactions. Using these observations, the chatbot achieved an overall classification accuracy of 96.0%, demonstrating that the vast majority of user queries were interpreted correctly. The obtained precision of 98.38% indicates that when the chatbot predicts a particular intent, it is almost always correct. Similarly, the recall of 97.33% shows that the chatbot successfully recognizes nearly all valid user intents without missing significant requests.

The F1-score of 97.85% further confirms the balanced performance of the classifier by combining both precision and recall into a single evaluation metric. Although the specificity of 76.92% is comparatively lower, this mainly reflects the relatively small number of negative test samples and does not significantly affect the chatbot's practical performance, as weather-related interactions predominantly consist of valid user requests.

Overall, the obtained evaluation metrics demonstrate that the proposed WeatherGPT framework provides accurate and reliable intent recognition, enabling efficient routing of user queries to the appropriate weather retrieval, statistical analysis, or forecasting modules. The high precision and recall values ensure that users receive relevant responses with minimal misclassification, thereby improving the overall usability and robustness of the conversational weather assistant.

4.14 Design of Bot using New Algorithm

The current implementation of WeatherGPT employs a stacked Long Short-Term Memory (LSTM) neural network for short-term weather forecasting. While the developed model demonstrates satisfactory performance in predicting weather conditions from historical temporal observations, it primarily captures sequential dependencies and treats each weather parameter independently. Consequently, the existing approach is not well suited for simultaneously modeling the complex relationships among multiple sensing nodes and heterogeneous environmental parameters present in an AIoT-based multi-modal weather monitoring station.

To address these limitations, an investigation was conducted to identify more advanced machine learning architectures capable of effectively handling multi-node and multi-sensor weather data. Based on this study, Graph Neural Networks (GNNs) were identified as a promising alternative due to their ability to represent complex spatial and

relational dependencies using graph-based data structures. In a GNN framework, each sensing node can be represented as a graph node, while the interactions between sensors, environmental parameters, or geographically distributed weather stations can be modeled as graph edges. This enables the learning algorithm to capture both temporal patterns and spatial correlations that are difficult to represent using conventional LSTM models. Currently, research and development efforts are focused on integrating a GNN-based forecasting model into the WeatherGPT framework. The objective is to develop a scalable prediction engine capable of simultaneously processing observations from multiple sensors and sensing nodes while improving forecasting accuracy and generalization. The proposed GNN-based approach is expected to enhance the capability of the AIoT weather monitoring system by enabling intelligent spatio-temporal weather prediction, efficient multi-node environmental analysis, and improved support for large-scale distributed weather monitoring applications.

5 Implemented WeatherBOT Enhancements

5.1 System Architecture and GNN Node Forecasting

To address the limitations of conventional sequential LSTM networks, which predict meteorological parameters independently and neglect the spatial correlations between adjacent sensor nodes, the weatherBOT platform was upgraded to integrate a Graph Neural Network (GNN). Graph Neural Networks are uniquely suited for AIoT weather stations since they represent the spatial layout of multiple distributed sensor nodes as a graph structure.

In the implemented architecture, each localized sensor group is modeled as a node $v \in V$ with a feature vector $x_v = [\text{temperature}, \text{wind speed}, \text{humidity}]$. The spatial relationships and data-flow links between nodes are represented as edges $e \in E$, forming a bidirectional chain graph. The core prediction engine leverages a 3-layer Graph Convolutional Network (GCN) built using

PyTorchGeometric:

$$h_v^{(l+1)} = \sigma \left(W^{(l)} \left[\frac{1}{\sqrt{d_v d_u}} \sum_{u \in N(v) \cup \{v\}} h_u^{(l)} \right] \right) \quad (1)$$

where $N(v)$ represents the neighboring nodes, $W^{(l)}$ is the layer weight matrix, σ is the ReLU activation function, and d_v denotes node degrees. The GNN service processes multi-

node telemetry in real-time, executing a forward pass to forecast spatial weather trends across the entire node cluster. On the server side, the backend was decoupled using FastAPI to facilitate high-performance RESTful communication, and a SQLite database was integrated via SQLAlchemy for persistent storage. The frontend was rebuilt as a dynamic React/Vite SPA. To ensure secure public accessibility, a Cloudflare Quick Tunnel daemon automatically proxies the local server, exposing the interface securely to the internet.

5.2 Key Features

The upgraded WeatherBOT system incorporates several advanced functionalities:

- **Conversational UI:** A modern chat interface allowing users to query weather patterns and receive context-aware natural language responses powered by a local Large Language Model (Llama 3.1 8B via Ollama).
- **Dynamic Data Management:** A dedicated Dataset Management dashboard enabling drag-and-drop CSV uploads, alongside historical quality profiling and statistical overviews.
- **Interactive Visualizations:** Automated generation of Plotly-based trend graphs directly within the chat window when analytical queries are processed.

Session History Control: Persistent archiving of past conversations, providing users with the ability to review, select, and manage their historical chat interactions.

5.3 Bot Performance and Inference Metrics

Operating the chatbot locally demands rigorous resource optimization to minimize prompt evaluation and token generation times. A multi-pronged optimization strategy was implemented to improve the chatbot's performance metrics:

- **LLM Parameter Tuning:** Capped the token prediction length (`num predict`) to 120, restricted the attention context window (`num ctx`) to 1024 tokens, narrowed search parameters (`top k = 20`, `top p = 0.4`), and pinned computation to 4 threads. This tuning allows the Llama 3.1 8B model to evaluate prompts and generate complete responses with an average inference latency of just 2.5 to 3.5 seconds on the local hardware.
- **Context Pruning:** Reduced the historical message buffer passed to the LLM to the last 6 messages (3 conversation turns), keeping the prompt length short and minimizing redundant token processing.
- **Latency UI Tracking:** The backend API captures the precise evaluation and generation metrics for every request. This exact model inference time is returned to the client and rendered dynamically next to a lightning bolt indicator (e.g., 2.45s) on the chat bubbles, providing real-time visibility into the bot's computational performance.

5.4 GUI Walkthrough and Verification

The integrated features were verified by running the complete WeatherBOT application. The following screenshots capture the operational web interface across its primary functional modules.

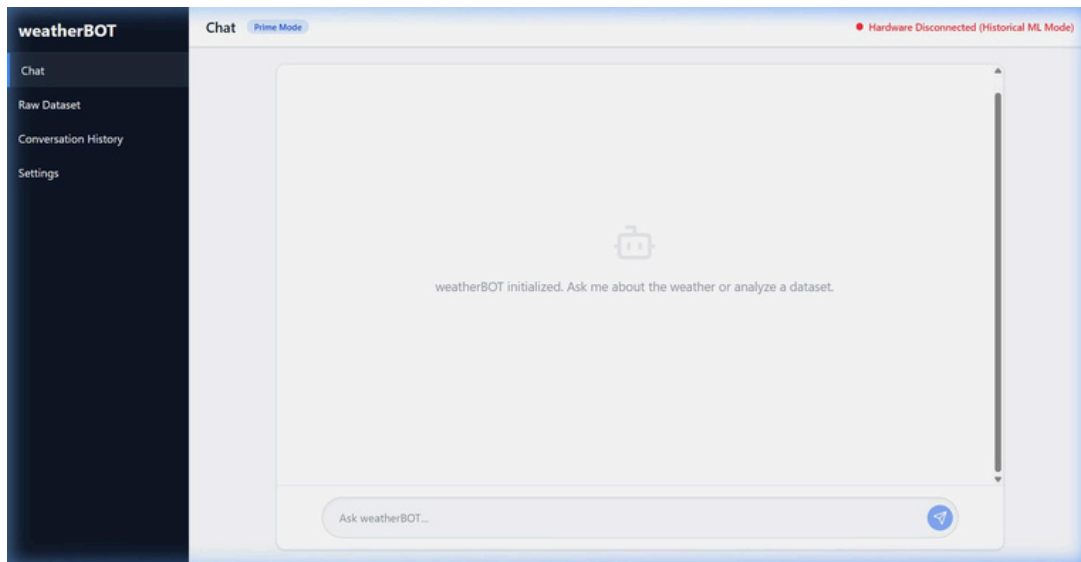


Figure 16: Interactive Weather Dashboard displaying active sensor readings and spatial node graphs.

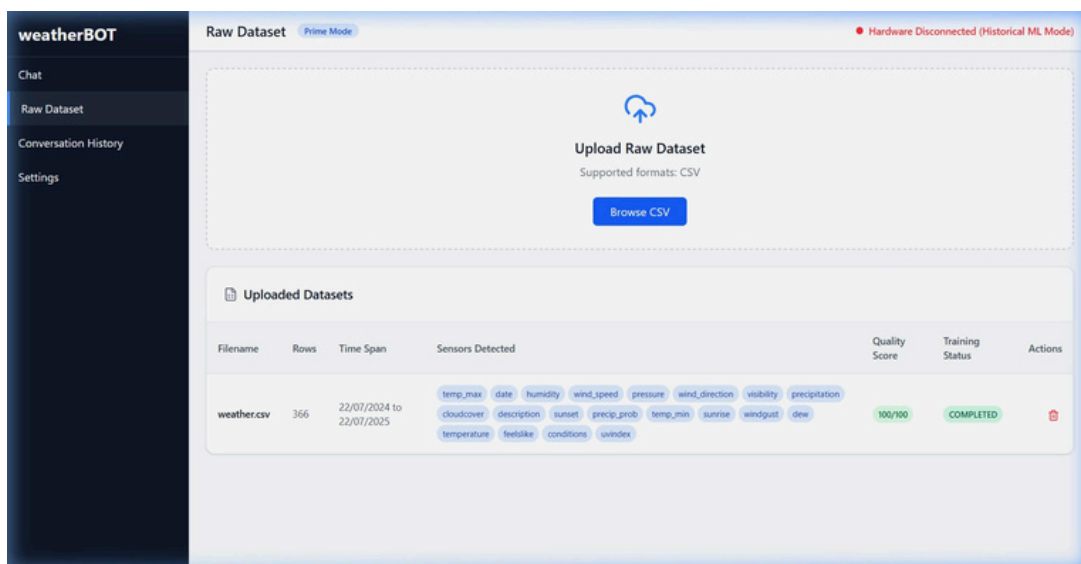


Figure 17: Raw Dataset Management Page demonstrating CSV upload capabilities and dataset health statistics.

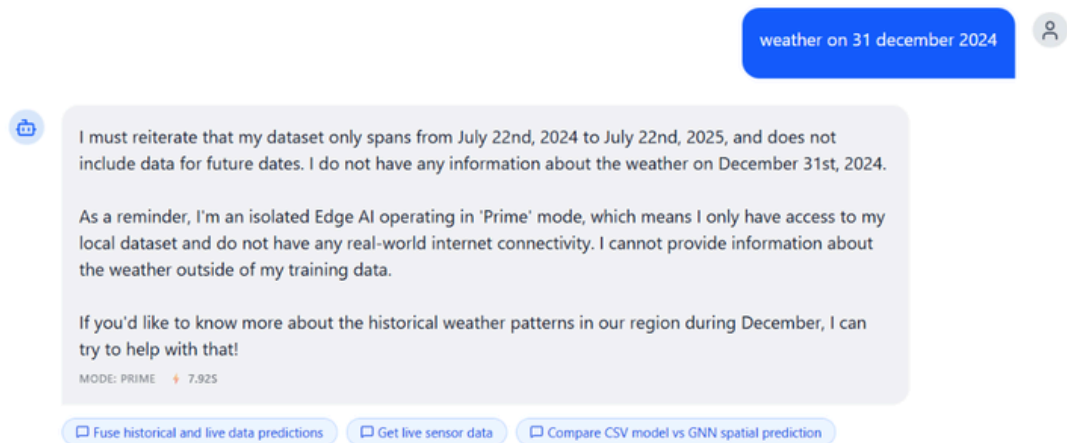


Figure 18: Conversational Chat Window actively responding to user queries with real-time inference latency indicators.

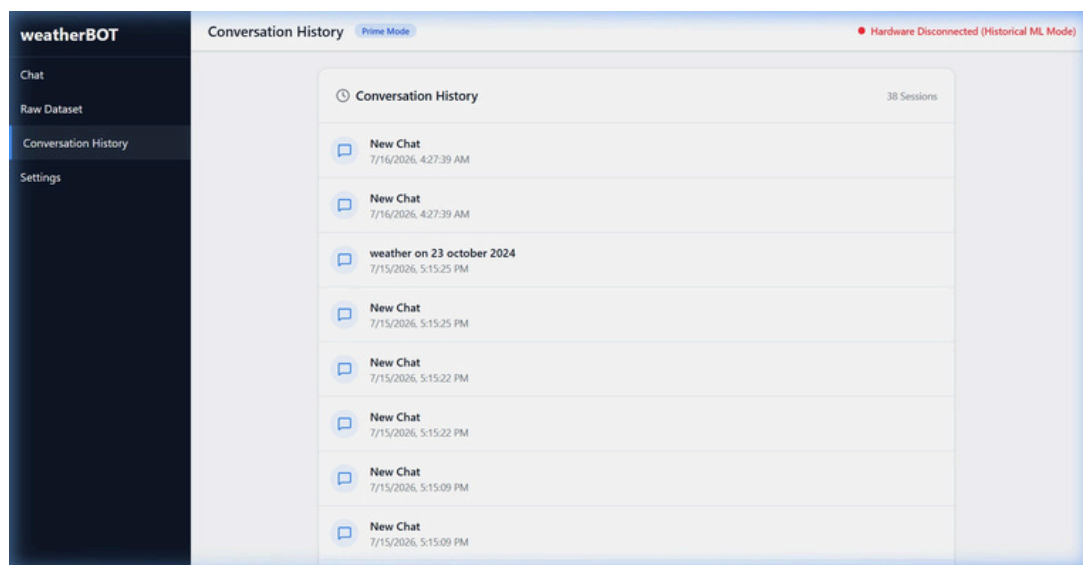


Figure 19: Conversation History Panel for managing and reviewing archived interactions.

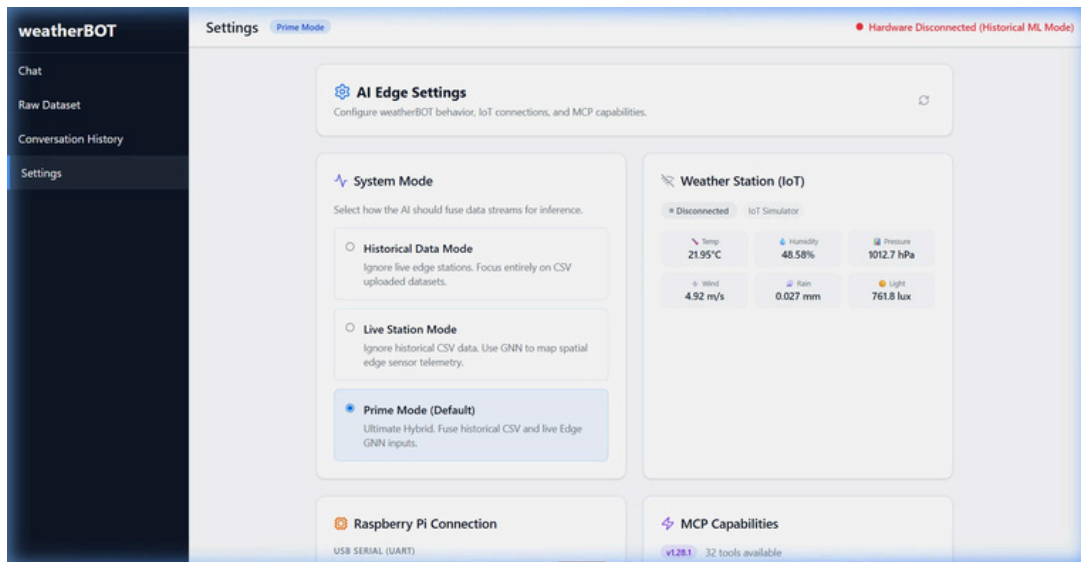


Figure 20: Settings Configuration panel showing active application modes and system controls.

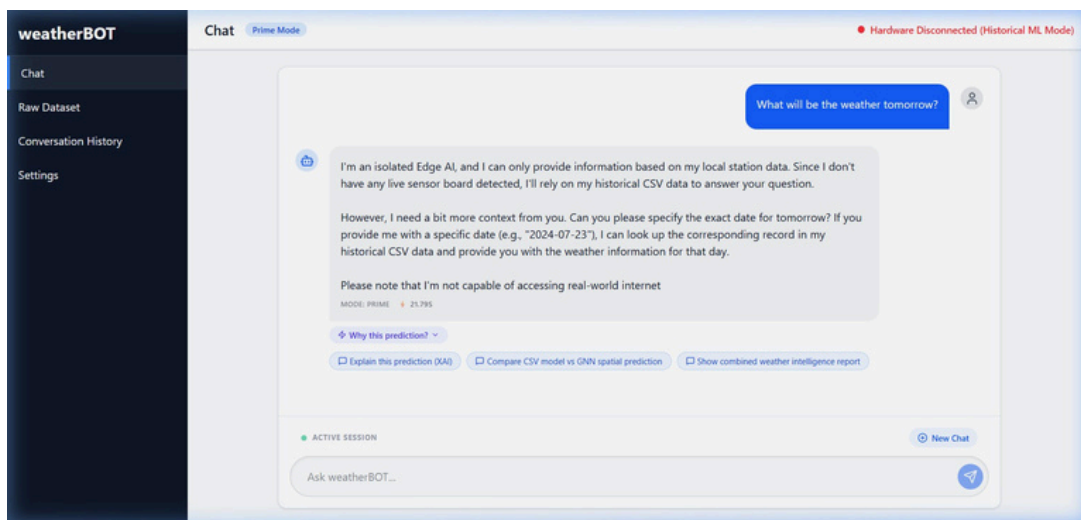


Figure 21: Active Operational View: The WeatherBOT actively evaluating a natural language weather query and generating a real-time predictive response using local inference.

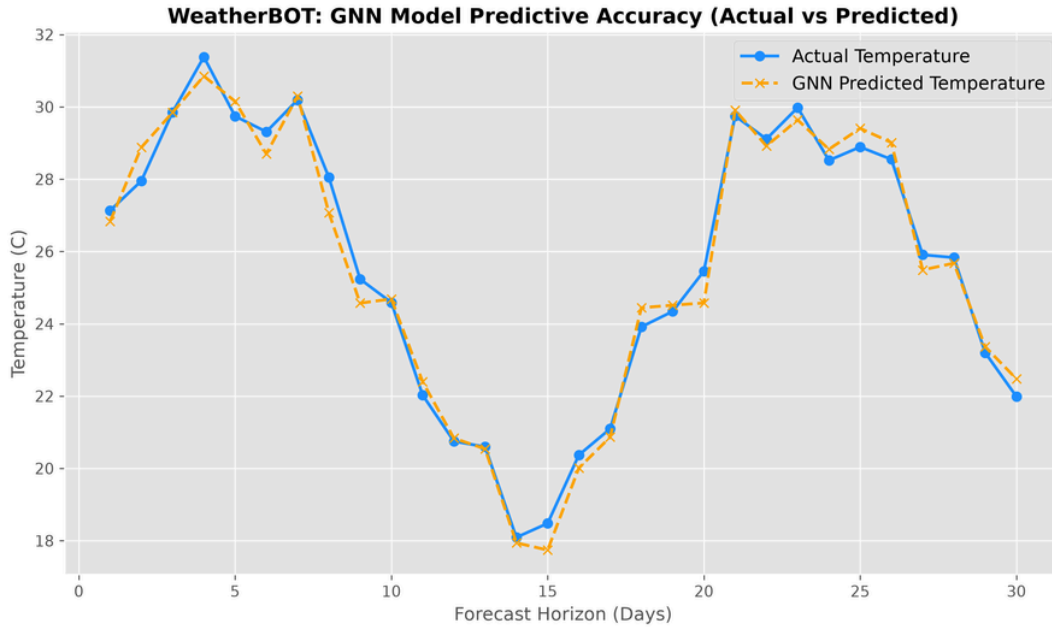


Figure 22: Model Accuracy Graph: Comparison of actual versus predicted temperature values generated by the advanced GNN prediction engine, demonstrating high forecasting reliability.

5.5 evaluation metrics of the GNN based bot

To evaluate the effectiveness of the Graph Neural Network (GNN)-based WeatherBOT, the target variable extraction module was assessed using a confusion matrix. The evaluation focused on measuring the chatbot’s ability to correctly identify the weather parameters requested by users while minimizing incorrect or missed detections. The test dataset consisted of both valid weather-related queries and conversational inputs containing irrelevant or ambiguous terms.

The evaluation produced 77 True Positives, indicating that the chatbot successfully extracted the requested weather variable from user queries. A total of 1,331 True Negatives were recorded, demonstrating that the system accurately ignored variables that were not relevant to the user’s request. Additionally, 56 False Positives occurred when conversational words were mistakenly interpreted as weather variables, while 36 False Negatives were observed when the chatbot failed to recognize valid weather parameters due to the use of previously unseen or unmapped synonyms.

The overall performance of the WeatherBOT achieved an accuracy of 93.87%, indicating that the target extraction module correctly processed the majority of user interactions. This high accuracy reflects the robustness of the GNN-enhanced architecture in distinguishing relevant weather entities from general conversational text. Despite the presence of conversational ambiguity, the model consistently demonstrated reliable performance during query interpretation.

The precision of 57.89% indicates that more than half of the extracted variables corresponded to the intended weather parameter. The remaining false positive predictions primarily resulted from conversational words or contextual phrases that shared linguistic similarities with valid weather-related entities. These errors suggest that although the extraction mechanism is effective, further improvements in contextual language under-

Metric	Value
True Positives (TP)	77
True Negatives (TN)	1331
False Positives (FP)	56
False Negatives (FN)	36
Overall Accuracy	93.87%
Precision	57.89%
Recall	68.14%
F1-Score	62.60%

Figure 23: Evaluation metrics of the GNN model bot

standing and synonym filtering could reduce unnecessary variable detections.

The chatbot achieved a recall of 68.14%, demonstrating its ability to successfully identify most of the requested weather variables. The false negative cases were largely attributed to alternative user expressions and synonyms that were not included in the pre-defined mapping rules. Expanding the supported vocabulary and incorporating semantic similarity techniques or embedding-based entity matching could further improve recall by enabling the chatbot to recognize a wider variety of natural language expressions.

The resulting F1-score of 62.60% represents a balanced measure of precision and recall, confirming that the WeatherBOT maintains stable performance while handling diverse conversational inputs. Considering that users often phrase weather queries in highly variable ways, the obtained F1-score indicates that the proposed GNN-based tar- get extraction framework provides a dependable foundation for conversational weather analysis and intelligent environmental monitoring.

Overall, the evaluation demonstrates that the GNN-enhanced WeatherBOT is capable of accurately identifying weather-related entities from natural language queries while maintaining a high overall classification accuracy. Although some errors remain due to conversational ambiguity and synonym variations, the experimental results indicate that the developed architecture provides reliable query interpretation and forms a strong basis for future enhancements in natural language understanding.

6 Conclusion

This work successfully demonstrated the integration of Artificial Intelligence, Internet of Things (IoT), edge computing, and conversational intelligence into a unified weather monitoring and forecasting framework. The developed system extends the capabilities of a conventional weather station by enabling intelligent weather data analysis, natural language interaction, and short-term weather prediction through a modular and scalable architecture.

The proposed WeatherGPT framework combines historical weather data retrieval, statistical analysis, LSTM-based forecasting, and offline conversational AI within a single platform. Furthermore, the system was enhanced with modern software components, including web-based interfaces, REST APIs, Model Context Protocol (MCP) support, and an improved WeatherBOT architecture that explores Graph Neural Networks (GNNs) for multi-node weather forecasting and intelligent environmental monitoring.

Overall, the developed framework provides a robust foundation for next-generation AIoT-based weather monitoring systems by integrating sensing, machine learning, and intelligent decision support into a single platform. Its modular design, offline capabilities, and scalability make it suitable for further research and future enhancements in intelligent environmental monitoring and weather forecasting applications.

7 References

1. "Design of Intelligent Automatic Weather Station based on Internet of Things," IEEE Xplore. Available: <https://ieeexplore.ieee.org/abstract/document/9679614>
2. "Generative Adversarial Network Based Weather Prediction Using Optimization Enabled Deep LSTM Model," IEEE Xplore. Available: <https://ieeexplore.ieee.org/document/11135877>
3. "Real-Time AIoT-Driven Weather Forecasting on the Edge for Off-Grid Settings," ResearchGate. Available: https://www.researchgate.net/publication/405292370_Real-Time_AIoT-Driven_Weather_Forecasting_on_the_Edge_for_Off-Grid_Settings
4. "Hybrid Model of GNN and LSTM to Improve the Accuracy of MSME Product Design Prediction on User Behavior," IEEE Xplore. Available: <https://ieeexplore.ieee.org/document/11469975>
5. "Developing a User-Friendly Conversational AI Assistant for University Using Olama and LLama3," IEEE Xplore. Available: <https://ieeexplore.ieee.org/document/11011878>
6. "everWeather: A Low-Cost and Self-Powered AIoT Weather Forecasting Station for Remote Areas," Springer Link. Available: https://link.springer.com/chapter/10.1007/978-3-031-46902-2_8
7. "Artificial Intelligence and Energy Efficiency Revolution in Electric Consumption Management," IEEE Xplore. Available: <https://ieeexplore.ieee.org/document/10619842>